```
SELECT AVG(marks) FROM student;
SELECT names FROM student WHERE marks > 87.6667;
```

RANKA
DATE / /

```
SELECT * FROM student WHERE city="Delhi";

SELECT MAX(marks)
  FROM (SELECT * FROM student WHERE city="Delhi") AS temp;
```

MySQL views

A view is a virtual table based on the result-set of an SQL statement.

A view always shows up-to-date data. The database engine recreate the view, everytime a user queries it.

```
CREATE VIEW view1 AS
  SELECT roll no, name FROM student;

SELECT * FROM view1;
```

Union:- It is used to combine the result set of two or more SELECT statements.

RANKA
DATE / /
PAGE

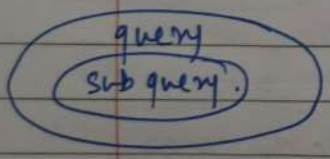
* every SELECT should have same no. of columns.
* columns must have same data types.
* Columns in every select should be in same order.

```
SELECT Column(s) FROM tableA
UNION
SELECT column(s) FROM tableB
```

UNION ALL → Executes even the duplicates that are in both tables.

SQL Subqueries

It is an inner query within the SQL query.
It involves 2 select statements




```
SELECT Column(s)
FROM table_name
WHERE Col_name operator (subquery);
```

Ex:- (i) Find avg. of class
(ii) find the name of student with marks > avg.
(iii) find students of Delhi
(iv) find their max marks using the sublist in step(iii)

roll no.	name	marks	city
101	anil	78	Pune
102	bhumika	93	Mumbai
103	chetan	85	Mumbai
104	dhruv	96	Delhi
105	emanuel	92	Delhi
106	farah	82	Delhi

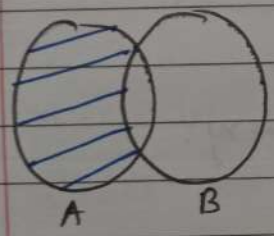
Full join:- Returns all records when there is a match in either left or right table.

Result

id	name	course
101	adam	null
102	bob	eng
103	casey	Science
105	null	maths
107	null	CSE

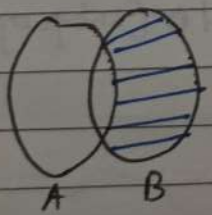
```
SELECT * FROM student AS a
LEFT JOIN course as b
  ON a.id= b.id;

UNION

SELECT * FROM student AS a
  RIGHT JOIN course AS b
    ON a.id= b.id;
```

Left Exclusive Join:-




```
SELECT * FROM student AS a
  LEFT JOIN course AS b
    ON a.id= b.id
    WHERE b.id is NULL;
```

Right Exclusive Join:-




```
SELECT * FROM student AS a
  RIGHT JOIN course AS b
    ON a.id= b.id
    WHERE a.id is NULL;
```

Self Join:- It is a regular join by table is joined with itself

```
SELECT *
FROM student as s
JOIN course as c
ON s.id=c.id
```

Inner join:- Returns records that have matching values in both tables

Student

student_id	name
101	adam
102	bob
103	casey

Course

student_id	course
102	eng
105	maths
103	science
107	CSE.

⇒ Result

student id	name	course
102	bob	eng
103	casey	science

```
SELECT *
  FROM students
INNER JOIN course
ON student.student_id = course.student_id;
```

Left join:- Returns all records from the left table and the matched records from the right table.

student

id	name
101	adam
102	bob
103	casey

Course.

id	course
102	eng
105	math
103	science
107	CSE

⇒ Result

id	name	course
101	adam	NULL
102	bob	eng
103	casey	science

```
SELECT *
  FROM students as s
LEFT JOIN course as c
  ON s.id = c.id;
```

Right join:- Returns all records from the right table, and the matched records from the left table.

Result

id	course	name
102	eng	bob
105	math	null
103	science	casey
107	CSE	null.

```
SELECT *
  FROM student as s
RIGHT JOIN course as c
  ON s.id = c.id;
```

Table Related Queries

Alter (to change the schema).
↳ design (column, datatypes, constraints)

RANKA
DATE / /

Add column:-
```
ALTER TABLE student;
ADD COLUMN percentage FLOAT;
```

Drop column:-
```
ALTER TABLE student;
DROP COLUMN marks;
```

Rename Table:-
```
ALTER TABLE student;
RENAME TO student_data;
```

Change Column (rename):-
```
ALTER TABLE student;
CHANGE age stu_age INT;
```
old clm name — new clm name — D-Type.

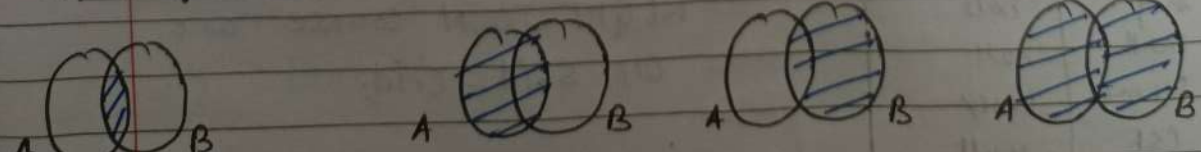
Modify Column:-
```
ALTER TABLE student;
~~CHANGE~~ MODIFY age VARCHAR(2);
```

Truncate:- Delete table's data not the table.
```
TRUNCATE TABLE student;
DROP student, // Delete the table.
```

Joins in SQL

Join is used to combine rows from two or more tables, based on a related column between them.

Types of joins:-



Inner join — Left join — Right join — Full join

Revisiting foreign key:-

RANKA
DATE / /
PAGE

```
CREATE TABLE dept (
        id INT PRIMARY KEY,
        name VARCHAR(50));

CREATE TABLE teacher (
        id INT PRIMARY KEY,
        name VARCHAR(50),
        dept_id INT,
    FOREIGN KEY (dept_id). REFERENCES dept(id));
```

(dept table)

Cascading for foreign key:-

On update cascade:- When we create a foreign key using UPDATE CASCADE the referencing rows are updated in the child table when the refrenced row is updated in the parent table which has a primary key.

On delete cascade:- When we create a foreign key with this option, it deletes the referencing row in the child table when the referenced row is deleted in the parent table which has a primary key.

```
CREATE TABLE student (
        id INT PRIMARY KEY,
        courseID INT,
        FOREIGN KEY (courseID) REFERENCES course(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE);
```

Group by clause:- Groups rows that have the same values into summary rows.

RANKA
DATE / /
PAGE

It collects data from multiple records and groups the result by one or more column.

SELECT City, COUNT(name) FROM STUDENT GROUP BY city;
// count no. of students in each city.

SELECT grade, COUNT(name) FROM student GROUP BY grade;

Having clause:- Applies some conditions on rows (where).
Applies any condition after grouping (having)

SELECT count(name), city FROM student GROUP BY city
HAVING MAX(marks) > 90;

Table related Queries

UPDATE student SET grade = "O" WHERE grade = "A";

UPDATE student SET grade = "B" WHERE ~~grade~~ marks
BETWEEN 80 AND 90;

UPDATE student SET marks = marks + 1;

DELETE FROM student WHERE marks < 33;

SELECT * FROM student WHERE marks+10>100;

SELECT * FROM student WHERE marks>90 and city="Mumbai";

SELECT * FROM student WHERE marks >90 or city="Mumbai";

SELECT * FROM student WHERE marks BETWEEN 80 AND 90;
(selects for a given range)

SELECT * FROM student WHERE city IN ("Delhi", "Mumbai");
(matches any value in the list)

SELECT * FROM student WHERE city NOT IN ("Delhi", "Mumbai");
(To negate the given condition)

Limit Clause :- sets an upper limit on number of (tuples) rows to be returned.

SELECT * FROM student LIMIT 3;

Order by Clause :- To sort in ascending (ASC) or descending (DESC) order.

SELECT * FROM student ORDER BY city ASC;
SELECT * FROM student ORDER BY roll no DESC;
(Top 3 students) SELECT * FROM student ORDER BY marks DESC LIMIT 3;

Aggregate functions :- They perform a calculation on a set of values, and return a single value.

COUNT(), MAX(), MIN(), SUM(), AVG().

SELECT max(marks) FROM STUDENT;

CHECK → it can create limit the values allowed in a column.

RANKA
DATE / /
PAGE

```
CREATE TABLE newTab (
        age INT CHECK (age>=18));
```

*
```
CREATE DATABASE collegedata;
USE collegedata;

CREATE TABLE student (
        roll no INT PRIMARY KEY,
        name VARCHAR(50),
        marks INT NOT NULL,
        grade VARCHAR(1),
        city VARCHAR(20) );

INSERT INTO student (roll no, name, marks, grade, city)
        VALUES( 101, "anil", 78, "C", "Pune"),
               (102, "bhumika", 93, "A", "Mumbai");
               (103, "chetan", 85, "B", "Mumbai");
               (104, "dhruv", 96, "A", "Delhi");
               (105, "emanuel", 12, "F", "Delhi");
               (106, "farah", 82, "B", "Delhi");

SELECT * FROM student;
SELECT name, marks FROM student;

SELECT DISTINCT city FROM student; // Remove duplicate values

SELECT * FROM student WHERE marks > 80;
SELECT * FROM student WHERE city = "Mumbai";
```
} WHERE clause.

* We can use with WHERE clause (+, -, *, /, %) (=, !=, >, >=, <, <=) / (AND, OR, NOT, IN, BETWEEN, ALL, LIKE, ANY) (&, |)

Keys:-

RANKA
DATE / /
PAGE

(i) Primary key:- It is a column (or set of column) in a table that uniquely identifies each row (a unique id).

There is only one Primary key and it should not be null.

(ii) foreign key:- It is a column (or set of columns) in a table that refers to the primary key in another table

foreign key can have duplicates and null values.

Constraints

SQL constraints are used to specify rules for data in a table.

NOT NULL → columns cannot have null value.

UNIQUE → all values in column are different

PRIMARY KEY → makes a column unique & not null but used only for one.

FOREIGN KEY → prevents actions that would destroy links between tables

Ex: CREATE TABLE temp (cust_id int,
FOREIGN KEY (cust_id) references customer(id));

DEFAULT → sets the default value of a column

Ex: salary INT DEFAULT 25000;

Ques:- Create a database of company XYZ

(i) Create a table inside the DB to store employee info (id, name, salary).

(ii) Add the following information:
1, 'Adam', 25000
2, 'Bob', 30000
3, 'Casey', 40000

(iii) Select & view all your table data.

```
CREATE DATABASE CompanyXYZ;
USE CompanyXYZ;

CREATE TABLE Companydata (id INT PREMARYKEY,
                          name VARCHAR(50),
                          Salary INT );

INSERT INTO companydata (id, name, salary) VALUES
                        ( 1, 'Adam', 25000,
                          2, 'Bob', 30000,
                          3, 'Casey', 40000);

SELECT * FROM companydata;

SELECT salary FROM Companydata WHERE salary>25000;

WIDTH 20 20 20 ; // changes width of the table
                    (in sqlite).

SELECT CURRENT_TIMESTAMP;
```

Signed and unsigned

RANKA
DATE / /
PAGE

Used in numeric datatypes:- INT, FLOAT, DOUBLE, TINYINT

unsigned → stores only +ve no.s
signed → stores +ve & -ve both.

TINYINT UNSIGNED (0 to 255) → Increase range for +ve no.s.

TINYINT (-128 to 127)

Types of SQL Commands :-

DDL (Data definition language) :- create, alter, rename, truncate, drop

DQL (Data query language) :- select

DML (Data Manipulation language) :- insert, update, delete

DCL (Data Control language) :- grant & revoke permission to users.

TCL (Transaction Control language) :- start transaction, commit, rollback.

```
CREATE DATABASE IF NOT EXISTS College;
DROP DATABASE IF EXISTS College;
```

Reduces chances of error if there is another database with same name.

```
SHOW DATABASES; // show all the databases
SHOW TABLES; // show all the tables.
```

ctrl + S → To save sql file. { Execute each line seperately. } { ↯ → Run sql file ; ↯ → Run highlighted command in sql. select each than run. }

Creating database :-

```
CREATE DATABASE db_name; // Create database

DROP DATABASE db_name;   // Delete database.
```

*
```
CREATE DATABASE college;
USE college;

CREATE TABLE student (
        id INT PRIMARY KEY,        ← for unique key
        name VARCHAR(50),
        age INT NOT NULL
        );

INSERT INTO student VALUES (1, 'AJAY', 20);
INSERT INTO student VALUES (2, 'AMAN', 19);

SELECT * FROM student; // Prints the table.
```

SQL Datatypes

Datatype	Description	Usage
CHAR	store characters of fixed length (0-255)	CHAR(50)
VARCHAR	" " " " " (0-255)	VARCHAR(50)
BLOB	string (0-65535) can store binary large object	BLOB(100)
INT	integer (-2,147,483,648 to 2147483647)	INT
TINYCNT	integer (-128 to 127)	TINYINT
BIGINT	integer (-9223372036854775808 to - - -)	BIGINT
BIT	can store x-bit values (0 < x < 64)	BIT(2)
FLOAT	Decimal no.	FLOAT
DATE	Date in format YYYY-MM-DD.	DATE
YEAR	year in four digit	YEAR
		BOOLEAN

Takes only required memory ↑ ... Take extra memory ↑

SQL

RANKA
DATE / /
PAGE

Database:- It is collection of data in a format that can be easily accessed.

* The software application used to manage our DB is called database management system.

Types of database

Relational

* Data stored in tables.

Ex: MySQL

Non-Relational.

* Data not stored in tables.

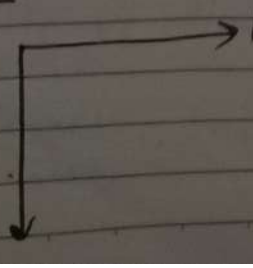
Ex:- MongoDB.

What is SQL (Structured Query language):-

* It is a programming language used to interact with relational database.

* It is used to perform CRUD operations:-
  - Create
  - Read
  - Update
  - Delete

Table:-



Columns → Tells design of the table structure / schema.

Rows → Tells individual data